A Symbolic Execution Approach to Bounded Verification of Abstract State Machines

Giuseppe Del Castillo[0009−0005−7020−6607]
giuseppedelcastillo@acm.org

Abstract. This paper describes a method for verifying properties of bounded prefixes of Abstract State Machine (ASM) runs. The method applies symbolic execution to convert an ASM program into a single "n-step" ASM rule that computes the state S_n reached from an initial state S_0 after n execution steps. The resulting rule has the structure of a decision tree whose internal nodes are conditional rules and whose leaves define symbolic update sets. Such "decision tree"-shaped rules can be used to verify whether a given invariant holds in all run prefixes of length at most n, or to derive a formula characterising all initial states that can lead to a violation of the invariant within n steps.
This paper is a substantially expanded and improved version of a short paper presented at ABZ 2024 and extends that work to the broader context of bounded verification.

1 Introduction

Abstract State Machines (ASM) [7] are a well-established, rigorous state-based method for system design and analysis. The ASM method has been used to define the semantics of programming and modelling languages (Prolog, Java and UML state and activity diagrams, among others), specify and document hardware and software systems (including processor architectures, embedded control systems, communication protocols) and mathematically prove properties of the modelled languages and systems. In some cases, interactive theorem provers were used to formally verify proofs of properties of ASM models (see the Prolog-WAM case study [20] or the Verifix project addressing the construction of formally verified compilers [14]). An overview of this line of research can be found in [8]. Support for automated verification has been provided in ASM tools through interfaces to model checkers like SMV / NuSMV or Spin [2, 11, 17] and SMT solvers [3, 4].

In this paper, we present a different approach to ASM verification, based on symbolic execution backed by an SMT solver. This method is limited to the verification of at most n steps of a sequential ASM run, where n is a fixed number ("bounded verification"), but is fully automatic, can be applied to ASM models with an infinite state space and generates human-readable simplified ASM rules, which can be inspected when looking for the cause of a property violation.[1] We also briefly introduce a tool implementing the techniques presented in this paper.

[1] The existing model checker based techniques are also fully automatic, but usually limited to finite state spaces. A notable exception is the partial support for nuXmv provided by the ASMETA tool set [6].

Symbolic execution is a classic technique that dates back to the 1970s [15, 22] and has started seeing renewed interest with the advent of SMT solvers, which have increased its practical applicability (see [5] for an overview). To the author's knowledge, the main works investigating symbolic execution of ASM are [16, 19, 21].[2] The work by Schellhorn et al. [21] is not about direct symbolic execution of ASM rules, but defines a relational encoding of rules into a logical formula and a logical calculus based on symbolic execution that can be used for verifying properties of ASM models using interactive theorem provers such as KIV. The work by Lezuo [16] is similar to ours in that it directly applies classic techniques of symbolic execution to ASM, but has a different focus, as it produces symbolic traces to perform translation validation on compiler-generated code, rather than a human-readable transformed ASM rule that can be executed. Finally, [19] applies symbolic execution to a timed variant of ASM for the purpose of estimating worst-case execution time.

The works by Arcaini et al. [3, 4] are related to ours in that they use an SMT solver for ASM verification. They define a scheme for translating an ASM into an equivalent logical representation processable by the SMT solver. In contrast, our approach invokes the SMT solver dynamically during symbolic execution to simplify guards of conditionals in order to prune infeasible execution paths.

This paper is structured as follows: after summarising basic notions of ASM (Sect. 2), we define the symbolic execution scheme (Sect. 3) and show how to use it for bounded verification of sequential ASM runs (Sect. 4). Finally, we briefly introduce a tool that implements the techniques presented in this paper and discuss some limitations of the method and potential improvements (Sect. 5).

2 Basic Notions of Abstract State Machines

In this section we summarise essential notions of single-agent, deterministic ASM as defined in [7] and introduce some notation. Only the subset of ASM needed to define the symbolic execution method is presented, in particular: only static, controlled and derived functions are allowed; there are no logical formulas, but terms of *Boolean* type are used instead; there is no *undef*; `choose` and `import` rules are not supported. Furthermore, we use a many-sorted variant of the ASM language including the possibility of defining parametric sorts, such as $Set(T)$ where T is any sort. We use the term "type" as a synonym for "sort" and do not provide any typing rules, as these are standard.[3] Besides the limitations men-

[2] Previous works such as the Verifix project [14] used symbolic execution techniques in an ad hoc manner, but did not address the systematic symbolic execution of ASM rules. Recently, [12] described a novel application of symbolic execution of control state ASM, but is not concerned with how to implement the symbolic execution.

[3] Strong typing is necessary for an effective use of SMT solvers to support symbolic execution. A type system for ASM that does not use *undef* to model partial functions, such as the one described in [23], while departing from the standard ASM definition, may facilitate the use of SMT solvers. Alternatively, it should be investigated how to fit *undef* into the type system without compromising the possibility of using SMT solvers efficiently. This problem is not addressed here and is left for future work.

tioned above and the use of types, the semantics given in this section completely follows [7], even though the presentation is in part slightly different.[4]

We adopt the following notational conventions: f is used for function names (of any type), P for Boolean function names (predicate names); v for variables; t for terms (of any type); $\bar{t}$ for a tuple of terms $(t_1, \ldots, t_n)$; G, φ or ψ for Boolean terms; R for (transition) rules, r for rule names, P for the special rule called the "program"; T for types; $\mathcal{T}$ for the carrier set of type T; x for values, i.e. elements of carrier sets; $\overline{x}$ for a tuple of values $(x_1, \ldots, x_n)$; $l = (f, \overline{x})$ for a location; Σ for signatures; S for states; η for variable assignments (also called *environments*); U for update sets. Finally, the notation $t : T$ means "t is of type T".

2.1 Computational Model

Abstract State Machines formalise a state-based model of computation, where computations (*runs*) are finite or infinite sequences of *states* $S_0 \to S_1 \to \ldots$ starting from an initial state S_0, where each state S_{i+1} is obtained by updating the state S_i according to an *update set* U_i, which is computed by evaluating in state S_i a closed "main" transition rule P, called the *program*. In symbols: $S_{i+1} = S_i + U_i$ where $U_i = \{\!|P|\!\}^{S_i}_{\emptyset}$, if U_i is consistent and non-empty.[5] A run *terminates* in a final state S_n if either

- P yields no updates in S_n, i.e. $\{\!|P|\!\}^{S_n}_{\emptyset}$ is an empty update set (termination with success), or
- P yields conflicting updates in S_n, i.e. $\{\!|P|\!\}^{S_n}_{\emptyset}$ is an inconsistent update set (termination with failure).

If there is no such final state, the run is an infinite sequence of states.[6] The meaning of states, updates, update sets and transition rules is defined below.

States The *states* are algebras over a given *signature* Σ (Σ-algebras for short). A signature consists of a set of *basic types* and a set of *function names*. Each function name has a fixed arity n and function type $T_1, \ldots, T_n \to T$, where the T_i and T are basic types (written $f : T_1, \ldots, T_n \to T$; if f has arity 0, $f :\to T$ is shortened to $f : T$). A Σ-algebra (state) S consists of:

[4] The most noticeable difference is that, given the absence of non-deterministic `choose` rules, the semantics of rules is defined as a *function* mapping rules to update sets, rather than as a *relation* (the "yields" relation of [7]).

[5] The unusual notation $\{\!|\cdot|\!\}$ is used here to refer to the semantics of terms and rules (evaluation) because the notation $[\![\cdot]\!]$ will be used later to denote *symbolic* evaluation. The semantics of a term t in state S under variable assignment η is a value $x = \{\!|t|\!\}^S_\eta$. Similarly, the semantics of a rule is an update set $U = \{\!|R|\!\}^S_\eta$. For the program P we take $\eta = \emptyset$, as P is closed. Definitions of $\{\!|\cdot|\!\}^S_\eta$ for terms and rules are given below.

[6] This is a natural notion of termination for deterministic, sequential systems. Other types of systems may require other termination criteria. For reactive systems, which normally do not terminate, one may stipulate that an empty update set—even in the absence of changes in the monitored functions defining the environment—does not cause termination, but an *empty move* (a transition from S_i to an identical S_{i+1}).

(i) a nonempty set $\mathcal{T}^S$ for each basic type T in Σ (the *carrier set* of basic type T in S; any element $x \in \mathcal{T}$ is called a *value* of type T, written $x : T$), and
(ii) a function $\mathbf{f}_S : \mathcal{T}_1^S \times \ldots \times \mathcal{T}_n^S \to \mathcal{T}^S$ for each function name $f : T_1, \ldots, T_n \to T$ in Σ (the *interpretation* of function name f in S).

A function name f in Σ can be declared to be of a given *kind*, which can be[7]:

- *static*, meaning that the interpretation of f is fixed and never changes during a run,
- *controlled*, meaning that the interpretation of f can change during a run due to updates resulting from the execution of transition rules (see below),
- *monitored*, meaning that the interpretation of f can change during a run in a way that is determined by the environment, or
- *derived*, meaning that the interpretation of f is defined by a computation rule in terms of other (static, controlled, or monitored) functions and can change during a run, but only as a consequence of changes in those functions.

Any signature Σ must contain basic types *Boolean* and *Set*(T) for any basic type T, static nullary function names (constants) *true* : *Boolean*, *false* : *Boolean*, the usual Boolean operators ($\neg$, $\wedge$, $\vee$, etc.) and equality ($= : T \times T \to$ *Boolean* for any T in Σ). Typically, Σ also includes at least a basic type *Integer* with the usual arithmetic operations, as well as any other type necessary for modelling the application at hand. Finally, a signature Σ may also contain a (possibly infinite but countable) number of *special constants*, i.e. static nullary function names of basic types denoting certain elements of the respective carrier set according to the usual conventions (e.g. $0, 1, \ldots$: *Integer*, $\emptyset_T$: *Set*(T)). These "background" types and function names have their standard interpretation in any state S.

When no ambiguity arises, the state S is not explicitly mentioned. In particular, we write $\mathcal{T}$ instead of $\mathcal{T}^S$ for carrier sets and $\mathbf{f}$ instead of $\mathbf{f}_S$ for static functions, as these never change during a run. The union of all carrier sets $\mathcal{T}$ is called the *superuniverse* $\mathcal{U}$ and any element $x \in \mathcal{U}$ is called a *value*.

Locations If $f : T_1, \ldots, T_n \to T$ is a function name, a pair $l = (f, \overline{x})$ with $\overline{x} = (x_1, \ldots, x_n) \in \mathcal{T}_1 \times \ldots \times \mathcal{T}_n$ is called a *location*. Then, the *type* of l is T and the *value* of l in a state S is the element $x \in \mathcal{T}$ with $x = \mathbf{f}_S(\overline{x})$.

Updates and Transitions A transition transforms a state S into its *sequel state* S' by changing the interpretation of some controlled function names on a finite number of points (i.e., updating the values of a finite number of locations).

More precisely, a transition transforms S into $S' = S+U$ by firing a consistent update set U at S. An *update set* U is a finite set of *updates* of the form $((f, \overline{x}), y)$,

[7] The ASM method, as defined in [7], foresees further function kinds, such as *shared*, which are not supported by the symbolic execution method of this paper. Note that monitored functions are also not directly supported by the method, but in Sect. 5.2 some indications are given as to how they can be dealt with by means of a mapping.

where $(f, \overline{x})$ is the location to be updated and y is the new value of the location after the update. An update set U is called *consistent* if it does not contain any updates $((f, \overline{x}), y)$ and $((f, \overline{x}), y')$ with $y \neq y'$ (*conflicting updates*).

If U is a *consistent* update set, the sequel state $S' = S + U$ of S is the state S' in which the carrier sets are as in S and, for each controlled function name $f : T_1, \ldots, T_n \to T$ and $\overline{x} \in \mathcal{T}_1 \times \ldots \times \mathcal{T}_n$, the interpretation of f in S' is

$$\mathbf{f}_{S'}(\overline{x}) \;=\; \mathbf{f}_{S+U}(\overline{x}) = \begin{cases} y & \text{if } ((f, \overline{x}), y) \in U \\ \mathbf{f}_S(\overline{x}) & \text{otherwise.} \end{cases}$$

If U is *inconsistent*, there is no sequel state S' (the run terminates with failure).

2.2 ASM Language

Terms Terms over a signature Σ include, as in (many-sorted) first-order logic, variables v and function applications $f(t_1, \ldots, t_n)$ for all t_i of the appropriate type (according to the type of f in Σ). If $n = 0$, $f()$ is abbreviated as f.

Due to their importance for symbolic execution, we add explicit *conditional terms*: for any terms G : *Boolean* ("guard") and t_1, t_2 of a same type T, there is a term "`if` G `then` t_1 `else` t_2" of type T. Finally, the term language includes let terms and quantifications $Q\, v \in t : \varphi$ over finite sets, where Q is $\forall$ or $\exists$, $v : T$ ranges over t : *Set*(T) and φ is a Boolean term in which v typically occurs free.

To summarise, the syntax of terms is as follows:

$t ::=$	$f(t_1, \ldots, t_n)$	(function application)
$\mid$	`if` G `then` t_1 `else` t_2	(conditional term)
$\mid$	v	(variable)
$\mid$	`let` v `=` t_1 `in` t_2	(let term)
$\mid$	$\forall v \in t : \varphi$	(universal quantification)
$\mid$	$\exists v \in t : \varphi$	(existential quantification)

The semantics of a term $t : T$ in state S is a $[\![t]\!]^S_\eta \in \mathcal{T}$ computed as in Table 1.

There is no distinct syntactic category for logical formulas in the ASM subset considered here. Instead, terms of *Boolean* type are used as formulas.

Transition rules The syntax of rules is as follows:

$R ::=$	`skip`	(empty rule)
$\mid$	$f(t_1, \ldots, t_n) := t$	(update rule)
$\mid$	`if` G `then` R_1 `else` R_2	(conditional rule)
$\mid$	R_1 `par` R_2	(parallel composition rule)
$\mid$	R_1 `seq` R_2	(sequential composition rule)
$\mid$	`iterate` R	(iteration rule)
$\mid$	`let` v `=` t `in` R	(let rule)
$\mid$	`forall` $v \in t$ `with` φ `do` R	(forall rule)
$\mid$	$r(t_1, \ldots, t_n)$	(call rule)

The semantics of a rule R in a state S is an update set $U = [\![R]\!]^S_\eta$ determined

Table 1. ASM semantics: evaluation of terms and rules

Terms

$$\frac{[\![t_i]\!]^S_\eta = x_i \quad \text{for all } i \in \{1,\ldots,n\}}{[\![f(t_1,\ldots,t_n)]\!]^S_\eta = \mathbf{f}_S(x_1,\ldots,x_n)} \qquad \frac{\eta(v) = x}{[\![v]\!]^S_\eta = x}$$

$$\frac{[\![G]\!]^S_\eta = \mathbf{true}}{[\![\texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2]\!]^S_\eta = [\![t_1]\!]^S_\eta} \qquad \frac{[\![G]\!]^S_\eta = \mathbf{false}}{[\![\texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2]\!]^S_\eta = [\![t_2]\!]^S_\eta}$$

$$\frac{[\![t_1]\!]^S_\eta = x}{[\![\texttt{let } v = t_1 \texttt{ in } t_2]\!]^S_\eta = [\![t_2]\!]^S_{\eta[v\mapsto x]}}$$

$$\frac{[\![t]\!]^S_\eta = \{x_1,\ldots,x_n\}}{[\![\forall v \in t : \varphi]\!]^S_\eta = \bigwedge_{i=1}^n [\![\varphi]\!]^S_{\eta[v\mapsto x_i]}} \qquad \frac{[\![t]\!]^S_\eta = \{x_1,\ldots,x_n\}}{[\![\exists v \in t : \varphi]\!]^S_\eta = \bigvee_{i=1}^n [\![\varphi]\!]^S_{\eta[v\mapsto x_i]}}$$

Rules

$$\frac{[\![t_i]\!]^S_\eta = x_i \quad \text{for all } i \in \{1,\ldots,n\}}{[\![f(t_1,\ldots,t_n) := t]\!]^S_\eta = \{((f,(x_1,\ldots,x_n)),[\![t]\!]^S_\eta)\}} \qquad \frac{}{[\![\texttt{skip}]\!]^S_\eta = \emptyset}$$

$$\frac{[\![G]\!]^S_\eta = \mathbf{true}}{[\![\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2]\!]^S_\eta = [\![R_1]\!]^S_\eta} \qquad \frac{[\![G]\!]^S_\eta = \mathbf{false}}{[\![\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2]\!]^S_\eta = [\![R_2]\!]^S_\eta}$$

$$\frac{[\![R_1]\!]^S_\eta = U_1 \qquad [\![R_2]\!]^S_\eta = U_2}{[\![R_1 \texttt{ par } R_2]\!]^S_\eta = U_1 \cup U_2}$$

$$\frac{[\![R_1]\!]^S_\eta = U_1 \qquad U_1 \text{ is inconsistent}}{[\![R_1 \texttt{ seq } R_2]\!]^S_\eta = U_1}$$

$$\frac{[\![R_1]\!]^S_\eta = U_1 \qquad U_1 \text{ is consistent} \qquad [\![R_2]\!]^{S+U_1}_\eta = U_2}{[\![R_1 \texttt{ seq } R_2]\!]^S_\eta = U_1 \oplus U_2}$$

$$\frac{[\![R]\!]^S_\eta = \emptyset}{[\![\texttt{iterate } R]\!]^S_\eta = \emptyset} \qquad \frac{[\![R]\!]^S_\eta \neq \emptyset}{[\![\texttt{iterate } R]\!]^S_\eta = [\![R \texttt{ seq } (\texttt{iterate } R)]\!]^S_\eta}$$

$$\frac{[\![t]\!]^S_\eta = x}{[\![\texttt{let } v = t \texttt{ in } R]\!]^S_\eta = [\![R]\!]^S_{\eta[v\mapsto x]}}$$

$$\frac{\{x \in [\![t]\!]^S_\eta \mid [\![\varphi]\!]^S_{\eta[v\mapsto x]} = \mathbf{true}\} = \{x_1,\ldots,x_n\}}{[\![\texttt{forall } v \in t \texttt{ with } \varphi \texttt{ do } R]\!]^S_\eta = \bigcup_{i=1}^n [\![R]\!]^S_{\eta[v\mapsto x_i]}}$$

$$\frac{[\![t_i]\!]^S_\eta = x_i \quad \text{for all } i \in \{1,\ldots,n\} \qquad r \text{ declared as } r(v_1,\ldots,v_n) = R}{[\![r(t_1,\ldots,t_n)]\!]^S_\eta = [\![R]\!]^S_{\eta[v_1\mapsto x_1,\ldots,v_n\mapsto x_n]}}$$

as shown in Table 1, wherein $U_1 \oplus U_2$ denotes the composition of update sets:

$$U_1 \oplus U_2 = U_2 \cup \{(l, x) \in U_1 \mid \text{there is no } x' \text{ with } (l, x') \in U_2\}$$

The binary rule composition operators `par` and `seq` are associative. Therefore, "R_1 `par` R_2 `par` $R_3 \ldots$" or "R_1 `seq` R_2 `seq` $R_3 \ldots$" do not need parentheses. It is nevertheless assumed that `par` and `seq` are left-associative when parsed. The following syntactic abbreviations can be used:

`if` G `then` R $\equiv$ `if` G `then` R `else skip`
`while` (G) R $\equiv$ `iterate` (`if` G `then` R)

3 Basic Symbolic Execution: Rule Simplification

In this section, we define a symbolic execution framework for ASM by extending the definition of terms and rules to *partially evaluated* terms and rules and by refining their evaluation accordingly. The refined, symbolic evaluation mappings can be applied to a modified ASM that has the same signature as the original ASM to be verified, but with some functions and/or locations left uninterpreted ("symbolic"). In this way, the ASM program is transformed into a simplified "decision tree" form, which is later used for verification.

Throughout this section we assume that all functions are static or controlled. Derived and monitored functions will be discussed in Sect. 5.2.

3.1 Uninterpreted Locations and Functions

The starting point for symbolic execution is that the value of some or all locations $(f, \overline{x})$ of one or more function names f may be *unknown* in a given (initial) state.[8] We call such locations *uninterpreted locations* (or *symbolic locations*).

If all locations $(f, \overline{x})$ of a given function name f are uninterpreted in S we say that f is *uninterpreted* (or *symbolic*).

Note that a location $(f, \overline{x})$ that is uninterpreted in S becomes interpreted in a sequel state $S' = S + U$ if there is an update of $(f, \overline{x})$ in U.

In this paper, we assume that only controlled function names, which represent the mutable state of the system being modelled, may be uninterpreted or have uninterpreted locations. Static function names, which define the operations on the data types forming the background of an ASM (such as *Boolean* or *Integer*) always have an interpretation in the sense of Sect. 2.1.

[8] We deliberately use the term "unknown" instead of "undefined" to avoid any confusion with the *undef* value normally used to model partial functions, which is an element of the ASM superuniverse. A location $(f, \overline{x})$ with $f(\overline{x}) = \mathit{undef}$ is not uninterpreted: its value is known and equals *undef*. The question may arise how the two can be distinguished. In practice, a tool may need to provide some syntax to specify which locations of f are symbolic locations with "unknown value". The current implementation does not yet support this, but only allows the declaration of functions that are uninterpreted as a whole: these are simply the functions that are declared in the signature, but for which no definition is provided.

3.2 Symbolic States and Environments

To formalise the scenario described above, where states may be only partially defined (i.e. include symbolic functions and/or locations), the notion of *symbolic state* is introduced. A *symbolic state* S over a signature Σ consists of

(i) a nonempty set $\mathcal{T}$ for each basic type T in Σ, and
(ii) a function $\mathbf{f}_{\mathsf{S}} : \mathcal{T}_1 \times \ldots \times \mathcal{T}_n \to PE\_TERM_{\Sigma}(T)$ for each function name $f : T_1, \ldots, T_n \to T$ in Σ (the *symbolic interpretation* of f in S),

where $PE\_TERM_{\Sigma}(T)$ is the set of *partially evaluated terms* of type T over Σ, defined in Sect. 3.3 below, which extends the set $TERM_{\Sigma}(T)$ of terms over Σ with *value terms* of the form $\langle \text{val } x \rangle$, for $x \in \mathcal{T}$, and *symbolic location terms* of the form $\langle \text{initial } (f, \overline{x}) \rangle$. Intuitively, $\langle \text{initial } (f, \overline{x}) \rangle$ stands for the unknown initial value of the symbolic location $(f, \overline{x})$.

Notationally, a symbolic state S is distinguished from a state S by using a different font.

A symbolic state $\mathsf{S} = \mathit{symstate}(S, \{f_1, \ldots, f_m\})$ can be derived from a state S by making some function names $\{f_1, \ldots, f_m\}$ from the signature Σ of S symbolic, while keeping the interpretation of the other function names of Σ unchanged:

$$\mathbf{f}_{\mathsf{S}}(x_1, \ldots, x_n) = \begin{cases} \langle \text{val } \mathbf{f}_S(x_1, \ldots, x_n) \rangle & \text{if } f \notin \{f_1, \ldots, f_m\} \\ \langle \text{initial } (f, (x_1, \ldots, x_n)) \rangle & \text{if } f \in \{f_1, \ldots, f_m\}. \end{cases}$$

Symbolic Environments Similarly, a variable assignment (environment) η mapping variables to values is replaced, in the symbolic execution framework, by a symbolic environment η mapping variables to partially evaluated terms.

3.3 Partially Evaluated Terms

Partially evaluated terms (abbreviated "pe-terms") have the same structure as terms, but the base case of their recursive definition includes, in addition to nullary function names and variables, the value terms $\langle \text{val } x \rangle$ and the symbolic location terms $\langle \text{initial } (f, \overline{x}) \rangle$ already mentioned above:

$$\begin{array}{lll} t ::= & \ldots & \text{(all previously defined terms, see Sect. 2.2)} \\ \mid & \langle \text{val } x \rangle & \text{(value term)} \\ \mid & \langle \text{initial } (f, \overline{x}) \rangle & \text{(symbolic location term)} \end{array}$$

where $x \in \mathcal{T}$ is a value from a carrier set $\mathcal{T}$ of the original ASM and $(f, \overline{x})$ is a location (as defined in Sect. 2.1) where $f : T_1, \ldots, T_n \to T$ is a controlled function name in Σ and $\overline{x} \in \mathcal{T}_1 \times \ldots \times \mathcal{T}_n$. The set of partially evaluated terms of type T is referred to as $PE\_TERM_{\Sigma}(T)$. It includes value terms $\langle \text{val } x \rangle$ for all carrier set elements $x \in \mathcal{T}$ and symbolic location terms $\langle \text{initial } (f, \overline{x}) \rangle$ for all controlled function names f with codomain T.

The idea behind the above definition is that, when uninterpreted functions or locations are present, a term cannot always be fully evaluated to a concrete value. Some subterms may reduce to values, others to symbolic locations, and

still others to forms that are neither values nor symbolic locations—or may not reduce at all. The goal is to simplify terms as much as possible. The definition of pe-terms is designed to reflect the structure of the symbolic evaluation's result.

Notationally, as $PE\_TERM_\Sigma(T)$ extends the regular terms $TERM_\Sigma(T)$ via the canonical inclusion $t \mapsto t$, we identify a term t and its corresponding pe-term, i.e. its image in $PE\_TERM_\Sigma(T)$, and refer to both simply as t.

3.4 Path Condition

The "path condition" is a standard notion in symbolic execution, characterised in [15] as the "accumulation of conditions which determines a unique control flow path through the program". In our setting, the path condition for an ASM subterm (or subrule) is the condition that holds in the branch of nested conditional terms (or rules) containing it. As this formula is a conjunction of the respective guards (for `then` branches) or negated guards (for `else` branches), we represent the path condition as a set of formulas Π, to be interpreted as conjunction.

In symbolic execution, a path condition is used as a constraint to guide the further execution of the given path. Accordingly, in our method, the evaluation of terms and rules does not only depend on a symbolic state S and variable assignment η, but also on a path condition Π.

3.5 Symbolic Evaluation of Partially Evaluated Terms

The *symbolic evaluation* ${}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta}$ of a (pe-)term t in a symbolic state S with a closing variable assignment η under a path condition Π is defined in Table 2 as a set of deduction rules ("evaluation rules").[9] Since η is a closing variable assignment for t, i.e. it contains bindings for all variables that occur free in t, t can be treated—in combination with η—as closed ("t is closed under η").

The goal of the symbolic evaluation mapping ${}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta}$ is to reduce (pe-)term t to a simplified t' in a "decision tree" form consisting only of

1. conditional terms at the top (if any);
2. (possibly) application terms $f(\bar{t})$, where f is a static function name, between the conditional terms at the top and the leaves;
3. pe-terms of the form $\langle \text{val } x \rangle$ or $\langle \text{initial } (f, \bar{x}) \rangle$ as leaves.

In the best case, t' is a single node $\langle \text{val } x \rangle$, meaning that t can be *fully evaluated*. In the worst case, t cannot be reduced to the required decision tree form, i.e. the symbolic execution fails. This is explicitly indicated by evaluation rules stating in their consequence that, for the given t, ${}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta}$ is not defined (*failure rules*).

One of the main means of obtaining the decision tree form specified above is the use of *conditional-lifting rules*. These are evaluation rules by which a

[9] The intention here is not to define a new logical calculus, but only to present the proposed symbolic execution method in a concise and precise manner. The formal definition of a notion of correctness for this method and the corresponding correctness proof are outside the scope of this paper and left for future work.

Table 2. Symbolic evaluation of pe-terms

$$\frac{}{{}^{\Pi}[\![\langle \text{val } x\rangle]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } x\rangle} \qquad \frac{}{{}^{\Pi}[\![\langle \text{initial } (f,\overline{x})\rangle]\!]^{\mathsf{S}}_{\eta} = \langle \text{initial } (f,\overline{x})\rangle}$$

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } x_i\rangle \ \text{ for all } i \in \{1,\ldots,n\}}{{}^{\Pi}[\![f(t_1,\ldots,t_n)]\!]^{\mathsf{S}}_{\eta} = \mathbf{f}_{\mathsf{S}}(x_1,\ldots,x_n)}$$

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G \texttt{ then } t_{i1} \texttt{ else } t_{i2} \qquad \text{for some } i \in \{1,\ldots,n\}}{{}^{\Pi}[\![f(t_1,\ldots,t_i,\ldots,t_n)]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G \texttt{ then } f(t_1,\ldots,t_{i1},\ldots,t_n) \texttt{ else } f(t_1,\ldots,t_{i2},\ldots,t_n)]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{f : T_1,\ldots,T_n \to T \quad T \neq \mathit{Boolean} \quad f \text{ is static} \quad \text{all } t'_i \neq \texttt{if } \ldots \quad {}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = t'_i \ \text{ for } i = 1,\ldots,n \quad t'_i \neq \langle \text{val } x_i\rangle \ \text{ for some } i \in \{1,\ldots,n\}}{{}^{\Pi}[\![f(t_1,\ldots,t_n)]\!]^{\mathsf{S}}_{\eta} = f(t'_1,\ldots,t'_n)}$$

$$\frac{f : T_1,\ldots,T_n \to \mathit{Boolean} \quad f \text{ is static} \quad \text{all } t'_i \neq \texttt{if } \ldots \quad {}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = t'_i \ \text{ for } i = 1,\ldots,n \quad t'_i \neq \langle \text{val } x_i\rangle \ \text{ for some } i \in \{1,\ldots,n\}}{{}^{\Pi}[\![f(t_1,\ldots,t_n)]\!]^{\mathsf{S}}_{\eta} = \mathsf{simplify\_formula}[\,\Pi, f(t'_1,\ldots,t'_n)\,]}$$

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = t'_i \quad f \text{ is not static} \quad t'_i \neq \langle \text{val } x_i\rangle \text{ for some } i \in \{1,\ldots,n\} \quad \text{all } t'_i \neq \texttt{if } \ldots}{{}^{\Pi}[\![f(t_1,\ldots,t_n)]\!]^{\mathsf{S}}_{\eta} \ \text{ is not defined}}$$

$$\frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \mathbf{true}\rangle \ \text{ or } \ {}^{\Pi}[\![t_1]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![t_2]\!]^{\mathsf{S}}_{\eta}}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![t_1]\!]^{\mathsf{S}}_{\eta}} \qquad \frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \mathbf{false}\rangle}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![t_2]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G' \texttt{ then } G_1 \texttt{ else } G_2}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G' \texttt{ then } (\texttt{if } G_1 \texttt{ then } t_1 \texttt{ else } t_2) \texttt{ else } (\texttt{if } G_2 \texttt{ then } t_1 \texttt{ else } t_2)]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = G' \qquad G' \notin \{\langle \text{val } \mathbf{true}\rangle, \langle \text{val } \mathbf{false}\rangle, \texttt{if } \ldots\} \qquad {}^{\Pi}[\![t_1]\!]^{\mathsf{S}}_{\eta} \neq {}^{\Pi}[\![t_2]\!]^{\mathsf{S}}_{\eta}}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G' \texttt{ then } {}^{\Pi\cup\{G'\}}[\![t_1]\!]^{\mathsf{S}}_{\eta} \texttt{ else } {}^{\Pi\cup\{\neg G'\}}[\![t_2]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{\eta(v) = t}{{}^{\Pi}[\![v]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta}} \qquad \frac{{}^{\Pi}[\![t_1]\!]^{\mathsf{S}}_{\eta} = t'_1}{{}^{\Pi}[\![\texttt{let } v \texttt{ = } t_1 \texttt{ in } t_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![t_2]\!]^{\mathsf{S}}_{\eta[v\mapsto t'_1]}}$$

$$\frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \{x_1,\ldots,x_n\}\rangle}{{}^{\Pi}[\![\forall v \in t : \varphi]\!]^{\mathsf{S}}_{\eta} = \textstyle\bigwedge_{i=1}^{n} {}^{\Pi}[\![\varphi]\!]^{\mathsf{S}}_{\eta[v\mapsto x_i]}} \qquad \frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \{x_1,\ldots,x_n\}\rangle}{{}^{\Pi}[\![\exists v \in t : \varphi]\!]^{\mathsf{S}}_{\eta} = \textstyle\bigvee_{i=1}^{n} {}^{\Pi}[\![\varphi]\!]^{\mathsf{S}}_{\eta[v\mapsto x_i]}}$$

$$\frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2 \qquad Q \text{ is } \forall \text{ or } \exists}{{}^{\Pi}[\![Qv \in t : \varphi]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G \texttt{ then } Qv \in t_1 : \varphi \texttt{ else } Qv \in t_2 : \varphi]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} \notin \{\langle \text{val } \ldots\rangle, \texttt{if } \ldots\} \qquad Q \text{ is } \forall \text{ or } \exists}{{}^{\Pi}[\![Qv \in t : \varphi]\!]^{\mathsf{S}}_{\eta} \ \text{ is not defined}}$$

conditional term occurring within another term is "moved up" and specialised versions of that term become branches of the conditional.

For each term form there are one or more evaluation rules, which are briefly discussed below.

Values and Symbolic Locations Two rules specify that value terms $\langle \text{val } x \rangle$ and symbolic location terms $\langle \text{initial } (f, \overline{x}) \rangle$ cannot be further reduced.

Function Application There are five rules for function application terms:

- The first rule is for the case where all arguments are fully evaluated. In this case, the result is either a value term $\langle \text{val } x \rangle$ or, if $(f, \overline{x})$ is uninterpreted, a symbolic location term $\langle \text{initial } (f, \overline{x}) \rangle$.
- The second rule is the conditional-lifting rule.
- The third rule is for the case of non-Boolean terms $f(\overline{t})$, where f is a static function and not all arguments can be fully evaluated. These terms are kept as they are, with their arguments evaluated as much as possible.[10]
- The fourth rule is for Boolean terms, which are treated specially due to the essential role they play in symbolic execution. Whenever the guard of a conditional can be fully evaluated, one branch can be eliminated and the number of paths to be explored can be reduced substantially. Therefore, particular efforts must be made to try to evaluate Boolean terms fully. The operator $\mathsf{simplify\_formula}[\,\Pi, \varphi\,]$ reduces, if possible, a Boolean term (formula) φ to $\langle \text{val } \mathbf{true} \rangle$ or $\langle \text{val } \mathbf{false} \rangle$ in view of the path condition Π:

$$\mathsf{simplify\_formula}[\Pi, \varphi] = \begin{cases} \langle \text{val } \mathbf{true} \rangle & \text{if it can be proved that } \bigwedge_{\psi \in \Pi} \psi \Rightarrow \varphi \\ \langle \text{val } \mathbf{false} \rangle & \text{if it can be proved that } \bigwedge_{\psi \in \Pi} \psi \Rightarrow \neg\varphi \\ \varphi & \text{otherwise} \end{cases}$$

  How $\mathsf{simplify\_formula}$ tries to prove the formulas depends on the implementation. In our tool, described in Sect. 5.2 below, this is done by a combination of: directly checking whether $\varphi \in \Pi$, $\neg\varphi \in \Pi$, applying Boolean algebra rewrite rules (such as $\varphi \wedge \mathit{false} = \mathit{false}$, or $\mathit{true} \wedge \varphi = \varphi$), and invoking an SMT solver if the previous measures do not succeed.[11]
- The fifth rule is the failure rule. The symbolic evaluation of $t = f(t_1, \ldots, t_n)$ fails if f is a controlled function name and not all subterms t_i can be fully evaluated. Such a term is not allowed in the target decision tree form, because it is not possible to unambiguously associate it with a specific location. Therefore, we stipulate that the transformation fails in this case.[12]

[10] These terms can be easily mapped to SMT solver terms, because static functions are "background" functions (see "States" subsection of Sect. 2.1 and last paragraph of Sect. 3.1) and correspond to the background functions of the SMT solver's theories.

[11] Experiments showed that, while the SMT solver is decisive for the final simplification, the rewrite rules contribute substantially to reducing the number of SMT solver calls needed (which can be quite costly in terms of run time).

[12] The reason for this restriction is to prevent the *aliasing* phenomenon, i.e., having different terms potentially referring to the same location.

Conditional There are four rules for conditional terms:

- The first and second rules cover the cases where the conditional term can be eliminated, either because the guard can be fully evaluated (to **true** or **false**) or because the two branches—after evaluation—are equal.
- The third rule is a conditional-lifting rule that is applied when the guard is itself a conditional.
- The fourth rule is applied when the guard G cannot be fully evaluated: in this case, the conditional remains in place, but its `then` and `else` branches are evaluated symbolically with the path condition extended by G and $\neg G$, respectively. Note that this is the only evaluation rule that directly modifies the path condition (other rules do this indirectly by constructing a conditional, which is then evaluated).

Variable-Binding Terms The rules for variables and let terms are straightforward and self-explanatory. With respect to quantifications:

- The first and second rules expand universal and existential quantifiers into conjunctions and disjunctions, respectively.[13]
- The third rule is the conditional-lifting rule.
- The fourth rule stipulates that the evaluation fails if the term defining the variable's range does not reduce to a concrete finite set value, because in this case the expansion rules cannot be applied.

Properties of the Mapping To conclude this section, we mention some useful and/or interesting properties of the symbolic evaluation mapping for pe-terms:

1. If there are no uninterpreted locations, the symbolic state $\mathsf{S} = \mathit{symstate}(S, \emptyset)$ coincides with S for any state S and the symbolic evaluation $[\![\cdot]\!]$ yields the same result as the non-symbolic evaluation $\{\!\{\cdot\}\!\}$, wrapped in $\langle \mathrm{val}\ \cdot \rangle$:
$$^{\emptyset}[\![\, t \,]\!]_{\eta}^{\mathit{symstate}(S,\emptyset)} = \langle \mathrm{val}\ \{\!\{t\}\!\}_{\eta}^{S} \rangle.$$
2. The symbolic evaluation mapping eliminates all variable-binding terms (let, quantifications) by instantiating the variables with ground terms. Since the term t to be evaluated is closed under the variable assignment η, the resulting pe-term $t' = {}^{\sqcap}[\![t]\!]_{\eta}^{\mathsf{S}}$ does not contain any variables (either free or bound), i.e., it is a ground pe-term.

The aliasing problem, i.e., determining whether two terms $f(\bar{t})$ and $f(\overline{t'})$ refer to the same location, is in general undecidable. Although methods for approximate alias analysis exist, they are quite complex and are only applicable to specific scenarios (such as arrays with integer indices).

Therefore, we decided to adopt a restrictive, but simple model. This limitation can be partially addressed using an "unfolding transformation" if the ranges of the arguments are finite, see Sect. 5.2.

[13] In Table 2, the $\bigwedge$ and $\bigvee$ symbols have a different meaning than in Table 1, as they denote the construction of terms with the $\wedge$ and $\vee$ function symbols rather than the application of the $\wedge$ and $\vee$ functions.

3. With the exception of the symbolic location terms, the reduced pe-term $t' = {}^{\pi}[\![t]\!]^{\mathsf{S}}_{\eta}$ consists only of static function applications (of which conditional terms are a special case) and constants (of which value terms are a special case). Therefore, the value of such a t' depends only on the values of the locations specified by the symbolic location terms that occur in it.

3.6 Symbolic Update Sets

For symbolic execution, update sets need to be replaced by symbolic update sets.

A *symbolic update set* U is defined as a finite set of *symbolic updates* of the form $((f, \overline{x}), t)$, where $(f, \overline{x})$ is the location to be updated and t is a ground pe-term in decision tree form to be associated to that location.

Transitions are defined accordingly. In the *sequel symbolic state* $\mathsf{S}' = \mathsf{S} + \mathsf{U}$ obtained by firing a symbolic update set U in symbolic state S, the symbolic interpretation of each controlled function name $f : T_1, \ldots, T_n \to T$ is

$$\mathbf{f}_{\mathsf{S}+\mathsf{U}}(\overline{x}) = \begin{cases} t & \text{if } ((f, \overline{x}), t) \in \mathsf{U} \\ \mathbf{f}_{\mathsf{S}}(\overline{x}) & \text{otherwise} \end{cases}$$

for each $\overline{x} \in \mathcal{T}_1 \times \ldots \times \mathcal{T}_n$. This is a simple adaptation of the basic definition based on regular (non-symbolic) update sets, which was given in Sect. 2.1.

The issue of consistency is more subtle. Unlike regular (non-symbolic) update sets, the consistency of a symbolic update set U cannot always be determined from U alone. If there is, for each location $l = (f, \overline{x})$ that occurs on the left-hand side of a (symbolic) update in U, only one update of l in U, then U is consistent under all circumstances. Otherwise, its consistency depends on the state S and on the path condition Π.[14] Therefore, we introduce a new notion of consistency.

A symbolic update set U is called *provably* (S, Π)*-consistent* if, for all symbolic updates $((f, \overline{x}), t)$ and $((f, \overline{x}), t')$ of the same location $(f, \overline{x})$ in U, the condition ${}^{\pi}[\![t = t']\!]^{\mathsf{S}}_{\emptyset} = \langle \text{val } \mathbf{true} \rangle$ holds. This is a conservative approximation of actual consistency: the definition requires that it can be proved that the formula $t = t'$ is always satisfied in symbolic state S under path condition Π.[15] Note that, if U is not provably (S, Π)-consistent, we must consider it potentially inconsistent and—in practice—treat it as inconsistent, i.e., causing termination with failure.

3.7 Partially Evaluated Transition Rules

Partially evaluated rules[16] ("pe-rules") have the same structure as rules, with an additional pe-rule of the form $\langle \text{upd } \mathsf{U} \rangle$ encapsulating a symbolic update set U:

[14] The environment η does not matter, as the pe-terms that appear on the right-hand sides of symbolic updates do not contain any variables.

[15] Nevertheless, this is a substantially better approximation than the one used in [10], where the symbolic update set was considered inconsistent whenever the conflicting t and t' were (structurally) different terms. With the new, improved definition, we can make use of the SMT solver to make the approximation as precise as possible.

[16] The term "rules" can stand for the ASM *transition rules* or for the *evaluation rules* of the symbolic execution scheme. We often use the term "rules" without further qualification, if it is clear from the context which kind of rules is meant.

$$
\begin{array}{lcll}
R & ::= & \ldots & \text{(all previously defined rules, see Sect. 2.2)} \\
 & | & \langle \mathsf{upd}\ \mathsf{U} \rangle & \text{(symbolic update set rule)}
\end{array}
$$

The idea behind this definition is that it is not always possible to fully evaluate a pe-rule to a symbolic update set. However, if some of its subrules can be fully evaluated, they can be replaced by the special pe-rule $\langle \mathsf{upd}\ \mathsf{U} \rangle$, which stands for the computed symbolic update set.

3.8 Symbolic Evaluation of Partially Evaluated Rules

The *symbolic evaluation* ${}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta}$ of (pe-)rule R in a symbolic state S with a closing variable assignment[17] η under path condition Π is defined in Tables 3, 4 and 5.

The goal is to reduce the (pe-)rule R to a simplified R' in a "decision tree" form consisting only of

1. conditional rules as inner nodes (if any);
2. pe-rules of the form $\langle \mathsf{upd}\ \mathsf{U} \rangle$, where U is a symbolic update set, as leaves.

Thus, for any R, ${}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta}$ is either a symbolic update set $\langle \mathsf{upd}\ \mathsf{U} \rangle$ or a conditional. The symbolic evaluation result ${}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta}$ is a decision tree representing all possible execution paths, as specified by the nested conditional in the inner nodes, and the symbolic update sets in the leaves correspond to the cumulative state changes resulting from the execution of the respective path from root to leaf.

The evaluation rules follow the same patterns already seen for the pe-terms, including the use of *conditional-lifting rules* and *failure rules* (see introductory paragraphs of Sect. 3.5).[18] Therefore, the rules will be discussed only briefly.

Two rules specify that `skip` and $\langle \mathsf{upd}\ \mathsf{U} \rangle$ cannot be further reduced.

The evaluation rules for *update rules* are similar to a subset of the rules for application terms. In particular, the first rule (fully evaluated case) and the last rule (failure rule) are similar in both scenarios. However, there are two conditional-lifting rules for update rules (instead of one), addressing the left-hand side (second rule) and the right-hand side (third rule), respectively.

The rules for *conditionals* are completely analogous to those for terms.

For `par` and `seq`, the rules for the fully evaluated case are derived in a rather direct manner from the respective (non-symbolic) semantic rules of Table 1. The remaining cases are the conditional-lifting rules for R_1 `par` R_2 and R_1 `seq` R_2, where R_1 and/or R_2 are conditional rules. The rules for these cases were obtained by using the following semantic equivalences (where *op* is `par` or `seq`):

$$
\begin{array}{l}
(\texttt{if}\ G_1\ \texttt{then}\ R_{11}\ \texttt{else}\ R_{12})\ op\ R_2 \simeq \texttt{if}\ G_1\ \texttt{then}\ (R_{11}\ op\ R_2)\ \texttt{else}\ (R_{12}\ op\ R_2) \\
R_1\ op\ (\texttt{if}\ G_2\ \texttt{then}\ R_{21}\ \texttt{else}\ R_{22}) \simeq \texttt{if}\ G_2\ \texttt{then}\ (R_1\ op\ R_{21})\ \texttt{else}\ (R_1\ op\ R_{22})
\end{array}
$$

Those rules rewrite, according to the above equivalences, the `par` or `seq` to a conditional rule, which is in turn evaluated symbolically.

---

[17] For explanations about the closing variable assignment, see Sect. 3.5, first paragraph.

[18] Because a conditional-lifting rule is needed whenever a conditional occurs within any transition rule, the resulting proliferation of rules can make the symbolic execution scheme appear cumbersome. A potential simplification is discussed in Sect. 5.2.

Table 3. Symbolic evaluation of pe-rules (part 1: basic rules)

$$\frac{}{{}^{\Pi}[\![\texttt{skip}]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \emptyset \rangle} \qquad \frac{}{{}^{\Pi}[\![\langle \text{upd } \mathsf{U} \rangle]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U} \rangle}$$

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } x_i \rangle \text{ for all } i \in \{1,\ldots,n\}}{{}^{\Pi}[\![f(t_1,\ldots,t_n) := t]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \{((f,(x_1,\ldots,x_n)), {}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta})\} \rangle}$$

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G \texttt{ then } t_{i1} \texttt{ else } t_{i2} \qquad \text{for some } i \in \{1,\ldots,n\}}{\begin{array}{l}{}^{\Pi}[\![f(t_1,\ldots,t_i,\ldots,t_n) := t]\!]^{\mathsf{S}}_{\eta} = \\ \quad {}^{\Pi}[\![\texttt{if } G \texttt{ then } f(t_1,\ldots,t_{i1},\ldots,t_n) := t \texttt{ else } f(t_1,\ldots,t_{i2},\ldots,t_n) := t]\!]^{\mathsf{S}}_{\eta}\end{array}}$$

$$\frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G \texttt{ then } t_1 \texttt{ else } t_2}{{}^{\Pi}[\![f(t_1,\ldots,t_n) := t]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G \texttt{ then } f(t_1,\ldots,t_n) := t_1 \texttt{ else } f(t_1,\ldots,t_n) := t_2]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = t_i' \quad {}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = t' \quad t_i' \neq \langle \text{val } x_i \rangle \text{ for some } i \in \{1,\ldots,n\} \quad \text{all } t_i', t' \neq \texttt{if} \ldots}{{}^{\Pi}[\![f(t_1,\ldots,t_n) := t]\!]^{\mathsf{S}}_{\eta} \text{ is not defined}}$$

$$\frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \mathbf{true} \rangle \text{ or } {}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![R_2]\!]^{\mathsf{S}}_{\eta}}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta}} \qquad \frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \mathbf{false} \rangle}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![R_2]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G' \texttt{ then } G_1 \texttt{ else } G_2}{\begin{array}{l}{}^{\Pi}[\![\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2]\!]^{\mathsf{S}}_{\eta} = \\ \quad {}^{\Pi}[\![\texttt{if } G' \texttt{ then } (\texttt{if } G_1 \texttt{ then } R_1 \texttt{ else } R_2) \texttt{ else } (\texttt{if } G_2 \texttt{ then } R_1 \texttt{ else } R_2)]\!]^{\mathsf{S}}_{\eta}\end{array}}$$

$$\frac{{}^{\Pi}[\![G]\!]^{\mathsf{S}}_{\eta} = G' \qquad G' \notin \{\langle \text{val } \mathbf{true} \rangle, \langle \text{val } \mathbf{false} \rangle, \texttt{if} \ldots\} \qquad {}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} \neq {}^{\Pi}[\![R_2]\!]^{\mathsf{S}}_{\eta}}{{}^{\Pi}[\![\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G' \texttt{ then } {}^{\Pi \cup \{G'\}}[\![R_1]\!]^{\mathsf{S}}_{\eta} \texttt{ else } {}^{\Pi \cup \{\neg G'\}}[\![R_2]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_1 \rangle \qquad {}^{\Pi}[\![R_2]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_2 \rangle}{{}^{\Pi}[\![R_1 \texttt{ par } R_2]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } (\mathsf{U}_1 \cup \mathsf{U}_2) \rangle}$$

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_1 \rangle \qquad {}^{\Pi}[\![R_2]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G_2 \texttt{ then } R_{21} \texttt{ else } R_{22}}{{}^{\Pi}[\![R_1 \texttt{ par } R_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G_2 \texttt{ then } (\langle \text{upd } \mathsf{U}_1 \rangle \texttt{ par } R_{21}) \texttt{ else } (\langle \text{upd } \mathsf{U}_1 \rangle \texttt{ par } R_{22})]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G_1 \texttt{ then } R_{11} \texttt{ else } R_{12} \qquad (R_2 \text{ is any rule})}{{}^{\Pi}[\![R_1 \texttt{ par } R_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G_1 \texttt{ then } (R_{11} \texttt{ par } R_2) \texttt{ else } (R_{12} \texttt{ par } R_2)]\!]^{\mathsf{S}}_{\eta}}$$

Table 4. Symbolic evaluation of pe-rules (part 2: `seq` and `iterate`)

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_1 \rangle \qquad \mathsf{U}_1 \text{ is not provably } (\mathsf{S},\Pi)\text{-consistent}}{{}^{\Pi}[\![R_1 \texttt{ seq } R_2]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_1 \rangle}$$

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_1 \rangle \qquad \mathsf{U}_1 \text{ is provably } (\mathsf{S},\Pi)\text{-consistent} \qquad {}^{\Pi}[\![R_2]\!]^{\mathsf{S}+\mathsf{U}_1}_{\eta} = \langle \text{upd } \mathsf{U}_2 \rangle}{{}^{\Pi}[\![R_1 \texttt{ seq } R_2]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } (\mathsf{U}_1 \oplus \mathsf{U}_2) \rangle}$$

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \mathsf{U}_1 \rangle \qquad {}^{\Pi}[\![R_2]\!]^{\mathsf{S}+\mathsf{U}_1}_{\eta} = \texttt{if } G_2 \texttt{ then } R_{21} \texttt{ else } R_{22}}{{}^{\Pi}[\![R_1 \texttt{ seq } R_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G_2 \texttt{ then } (\langle \text{upd } \mathsf{U}_1 \rangle \texttt{ seq } R_{21}) \texttt{ else } (\langle \text{upd } \mathsf{U}_1 \rangle \texttt{ seq } R_{22})]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta} = \texttt{if } G_1 \texttt{ then } R_{11} \texttt{ else } R_{12} \qquad (R_2 \text{ is any rule})}{{}^{\Pi}[\![R_1 \texttt{ seq } R_2]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{if } G_1 \texttt{ then } (R_{11} \texttt{ seq } R_2) \texttt{ else } (R_{12} \texttt{ seq } R_2)]\!]^{\mathsf{S}}_{\eta}}$$

$$\frac{{}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \emptyset \rangle}{{}^{\Pi}[\![\texttt{iterate } R]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \emptyset \rangle} \qquad \frac{{}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta} \neq \langle \text{upd } \emptyset \rangle}{{}^{\Pi}[\![\texttt{iterate } R]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![R \texttt{ seq } (\texttt{iterate } R)]\!]^{\mathsf{S}}_{\eta}}$$

Table 5. Symbolic evaluation of pe-rules (part 3: variable-binding rules)

$$\frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = t'}{{}^{\Pi}[\![\texttt{let } v \texttt{ = } t \texttt{ in } R]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta[v \mapsto t']}}$$

$$\frac{{}^{\Pi}[\![t]\!]^{\mathsf{S}}_{\eta} = \langle \text{val } \{x_1, \ldots, x_n\} \rangle}{{}^{\Pi}[\![\texttt{forall } v \in t \texttt{ with } \varphi \texttt{ do } R]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![\texttt{par}_{i=1}^{n}(\texttt{if } {}^{\Pi}[\![\varphi]\!]^{\mathsf{S}}_{\eta[v \mapsto x_i]} \texttt{ then } {}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta[v \mapsto x_i]})]\!]^{\mathsf{S}}_{\eta}}$$

where $\texttt{par}_{i=1}^{n} R_i = \begin{cases} \texttt{skip} & \text{if } n = 0 \\ R_1 & \text{if } n = 1 \\ (\texttt{par}_{i=1}^{n-1} R_i) \texttt{ par } R_n & \text{if } n \geq 2 \end{cases}$

Additionally, there are a conditional-lifting rule and a failure rule for the `forall` rule analogous to those defined for quantification terms in Table 2.

$$\frac{{}^{\Pi}[\![t_i]\!]^{\mathsf{S}}_{\eta} = t'_i \text{ for all } i \in \{1, \ldots, n\} \qquad r \text{ declared as } r(v_1, \ldots, v_n) = R}{{}^{\Pi}[\![r(t_1, \ldots, t_n)]\!]^{\mathsf{S}}_{\eta} = {}^{\Pi}[\![R]\!]^{\mathsf{S}}_{\eta[v_1 \mapsto t'_1, \ldots, v_n \mapsto t'_n]}}$$

The evaluation of the `iterate` rule is defined recursively in terms of `seq`.

Finally, for the variable-binding rules, the symbolic evaluation of the `let` rule and of the call rule are straightforward. The `forall` rule is treated by expanding it to a `par`-block in a similar manner as already seen for quantifiers (which were expanded to conjunctions or disjunctions).

Properties of the Mapping Similar to the case of pe-terms, the symbolic evaluation mapping for pe-rules has the following properties:

1. If there are no uninterpreted locations, $\mathsf{S} = \mathit{symstate}(S, \emptyset)$ coincides with S for any state S and ${}^{\emptyset}[\![R]\!]^{\mathsf{S}}_{\eta} = \langle \text{upd } \{\!\{R\}\!\}^{S}_{\eta} \rangle$.
2. A pe-rule R' that is the result of symbolic evaluation (i.e., $R' = {}^{\sqcap}[\![R]\!]^{\mathsf{S}}_{\eta}$ for some R) does not contain variables (in other words, R' is a ground pe-rule).
3. The update set of such a R' depends only on the values of the locations specified by the symbolic location terms that occur in it.

3.9 Conversion to ASM Transition Rule

After applying the transformation defined in Tables 3, 4 and 5, the generated pe-rule can be converted back to an equivalent ordinary ASM transition rule (as defined in Sect. 2.2), not containing any special $\langle \text{upd } \ldots \rangle$ pe-rules, nor any $\langle \text{val } \ldots \rangle$ and $\langle \text{initial} \ldots \rangle$ pe-terms. To achieve this, all $\langle \text{val } x \rangle$ pe-terms are replaced with ordinary terms that evaluate to x (e.g., *true*, *false* if x is Boolean, or special integer constants if x is an integer). All $\langle \text{initial } (f, (x_1, \ldots, x_n)) \rangle$ pe-terms are replaced by corresponding function application terms $f(t_1, \ldots, t_n)$, where the t_i are terms that evaluate to x_i (see above). All $\langle \text{upd } \mathsf{U} \rangle$ pe-rules are similarly replaced by corresponding `par` blocks of update rules, each of which corresponds to a respective symbolic update in U.

4 Bounded Verification for Sequential ASM Runs

A first version of the symbolic execution scheme defined in detail in Section 3 was introduced in [10]—for a more limited subset of ASM—as a method to transform ASM rules with `seq` and `iterate` into basic ASM rules.

In [9], its scope of application was extended to enable (bounded) verification of (invariant) properties of models of smart contracts defined as sequential ASMs.

This symbolic execution approach to bounded verification, which was only sketched in [9], is described more precisely in this section.

4.1 Symbolic Execution of Sequential ASMs

For an ASM with program P, the state S_n of a sequential ASM run $S_0, S_1, \ldots$ starting in the initial state S_0 (see definition in Sect. 2.1, first paragraph) equals the state reached by executing P^n in S_0, where P^n is the program P iterated n

times by means of the `seq` rule.[19] More precisely:

$$S_n = S_0 + \{\!\{P^n\}\!\}_{\emptyset}^{S_0}$$

where $P^0 := \texttt{skip}$, $P^1 := P$ and $P^{i+1} := P^i \texttt{ seq } P$ $(i \geq 1)$.

Therefore, to symbolically execute i steps of a sequential ASM run $(i \geq 0)$, we can use the symbolic evaluation rules defined in Sect. 3 to compute ${}^{\emptyset}[\![P^i]\!]_{\emptyset}^{\mathsf{S}_0}$ starting from an initial symbolic state S_0.

For a run prefix consisting of the first $n+1$ states $S_0, S_1, \ldots, S_n$ of the run, i.e., the first n steps, we may compute ${}^{\emptyset}[\![P^i]\!]_{\emptyset}^{\mathsf{S}_0}$ for $i = 0, \ldots, n$. However, for reasons of efficiency, instead of symbolically evaluating P^i from scratch for each i, we start from the previous symbolic evaluation result according to the following scheme: $\mathsf{P}^0_{\mathsf{S}_0} := \langle \text{upd } \emptyset \rangle$, $\mathsf{P}^1_{\mathsf{S}_0} := {}^{\emptyset}[\![P]\!]_{\emptyset}^{\mathsf{S}_0}$ and $\mathsf{P}^{i+1}_{\mathsf{S}_0} := {}^{\emptyset}[\![\mathsf{P}^i_{\mathsf{S}_0} \texttt{ seq } P]\!]_{\emptyset}^{\mathsf{S}_0}$ $(1 \leq i \leq n)$.

4.2 Invariant Checking

The above construction can be used as a basis for verifying properties of (finite prefixes of) ASM sequential runs. We only consider the verification of properties that hold in a single state or, by extension, in a number of states of a run independently of what happens in other states of the same run. In practice, meaningful properties of this kind which one may want to verify are often *invariants*, i.e., properties that should hold in all states of all runs.[20]

In the rest of this section, we define a method for *bounded verification* of invariants based on the symbolic execution techniques of this paper. Bounded verification means in this context that the verification is limited to those states that can be reached from an initial state within a fixed number n of steps.

The starting point are the pe-rules $\mathsf{P}^0_{\mathsf{S}_0}, \mathsf{P}^1_{\mathsf{S}_0}, \ldots, \mathsf{P}^n_{\mathsf{S}_0}$, as defined at the end of Sect. 4.1. Since all these pe-rules have been obtained as a result of the application of the symbolic evaluation mapping $[\![\cdot]\!]$, they are in "decision tree" form, i.e. they have conditional rules as inner nodes and symbolic update sets $\langle \text{upd } \mathsf{U} \rangle$ as leaves.

The recursive function ${}^{\Pi}ivcs^{\mathsf{S}_0}_{\varphi}(R)$—where *ivcs* stands for "invariant violation conditions"—traverses a pe-rule R in decision tree form and returns a set of all conditions which, when met in initial state S_0, may lead to a violation of invariant φ after the execution of R (while Π is an auxiliary parameter that keeps track of the current path condition as the conditionals are traversed):

$${}^{\Pi}ivcs^{\mathsf{S}_0}_{\varphi}(\texttt{if } G \texttt{ then } R_1 \texttt{ else } R_2) := ({}^{\Pi \cup \{G\}}ivcs^{\mathsf{S}_0}_{\varphi}(R_1)) \cup ({}^{\Pi \cup \{\neg G\}}ivcs^{\mathsf{S}_0}_{\varphi}(R_2))$$

$${}^{\Pi}ivcs^{\mathsf{S}_0}_{\varphi}(\langle \text{upd } \mathsf{U} \rangle) := \begin{cases} \emptyset & \text{if } {}^{\Pi}[\![\varphi]\!]_{\emptyset}^{\mathsf{S}_0 \oplus \mathsf{U}} = \langle \text{val } \mathbf{true} \rangle \\ \{\bigwedge_{\psi \in \Pi} \psi\} & \text{if } {}^{\Pi}[\![\varphi]\!]_{\emptyset}^{\mathsf{S}_0 \oplus \mathsf{U}} = \langle \text{val } \mathbf{false} \rangle \\ \{(\bigwedge_{\psi \in \Pi} \psi) \wedge \neg({}^{\Pi}[\![\varphi]\!]_{\emptyset}^{\mathsf{S}_0 \oplus \mathsf{U}})\} & \text{otherwise} \end{cases}$$

[19] This is well-known from ASM theory, see for example [7], Sect. 4.1.1 (in particular Lemma 4.1.2), as well as Lemma 2.4.3 on page 67 in combination with the semantics of `seq`, consistent case (see Table 1).

[20] We have not yet considered the problem of how to apply the methods presented here to the verification of temporal logic formulas expressing relations between states of a same run. This is a subject for future work.

(note that the computed set of formulas is to be understood as a disjunction).[21]

The set of conditions leading to the violation of φ after exactly i steps (for the given program P and initial symbolic state S_0) can then be computed by

$$ivcs_i := {}^{\emptyset}ivcs_{\varphi}^{\mathsf{S}_0}(\mathsf{P}^i_{\mathsf{S}_0}),$$

as $\mathsf{P}^i_{\mathsf{S}_0}$ corresponds to iterating i times the execution of P from initial state S_0.

If $ivcs_i = \emptyset$ for all $i \in \{0, \ldots, n\}$, then the invariant φ holds in all run prefixes of length at most n or, in other words, φ is never violated within n steps (including in the initial state, i.e., for $i = 0$). Otherwise, the formula

$$ivc_{0\ldots n} := \bigvee_{i=0}^{n}(\bigvee_{\psi \in ivcs_i} \psi)$$

characterises the concrete initial states that may lead to an invariant violation within n steps (a subset of all concrete states represented by symbolic state S_0).

Note that this method is a conservative approximation, in the sense that:

- it can produce *false negatives*, i.e., results indicating that an invariant may be violated, even though it is actually never violated in any state reachable from the initial state within n steps: what "invariant may be violated" means in this context is that it cannot be proved that it is not violated;
- it produces *no false positives*: if the result is that the invariant is not violated within n steps, then this is actually true in all states that can be reached from any initial state represented by the symbolic state S_0 within n steps.

As usual when working with methods of this kind, when inspecting negative results to understand their cause, one would check whether they are in fact false negatives. Often the false negatives can be eliminated by modifying the ASM model and/or the properties to be verified, e.g., by strengthening preconditions.

5 Implementation, Limitations, Potential Improvements

5.1 Implementation

The techniques presented in this paper have been implemented in a prototype tool, which is publicly available at

`https://github.com/constructum/asm-symbolic-execution`

The tool is implemented in F# (an ML dialect for the .NET framework) and uses the Z3 SMT solver [18] via the .NET interface provided by Z3. The concrete syntax used for specifying ASM models, as well as the properties to be verified, is a subset of AsmetaL [13].[22] With respect to the techniques of this paper, the tool provides:

[21] In order to relate the present definition to the terminology previously used in [9], it can be pointed out that, in the case distinction used to define ${}^{\Pi}ivcs_{\varphi}^{\mathsf{S}_0}(\langle \mathsf{upd}\ \mathsf{U} \rangle)$, the first case corresponds to the *met* case, the second case to *definitely violated*, the third case to *possibly violated* (all three relative to the given path condition Π).

[22] The language used in the first implementation, made for the ABZ 2024 paper [10], is a tiny subset of UASM [1]. In view of the current AsmetaL support, the use of this legacy language is deprecated and its support may be discontinued at some point.

- a `-steps` i option that constructs the pe-rules $\mathsf{P}^i_{\mathsf{S}_0}$ as defined in Sect. 4.1;
- a `-invcheck` n option to check whether the specified invariants hold within the first n steps: this is an implementation of Sect. 4.2.[23]

5.2 Limitations and Potential Improvements

Nested Non-Static Functions and Unfolding Transformation The main limitation of the symbolic execution method presented in Sect. 3 is its inability to process terms of the form $f(t_1, \ldots, t_n)$ or update rules $f(t_1, \ldots, t_n) := t$ when f is a controlled function name and not all subterms t_i can be fully evaluated (see also Sect. 3.5, "Function Application", explanation about 5th rule, and footnote 12).

One way to overcome this limitation would be to discriminate between two terms $f(\overline{t})$ and $f(\overline{t'})$ potentially causing aliasing by generating conditionals that check equality of $\overline{t}$ and $\overline{t'}$ and have a branch for the "equal" case where $f(\overline{t})$ and $f(\overline{t'})$ are mapped to the same location and a branch for the "not equal" case where they denote different locations (and let the symbolic evaluation rules prune the branches, if possible). However, this would require a more complex symbolic execution framework where locations are not simply pairs $(f, \overline{x})$, where $\overline{x}$ is a tuple of values. Furthermore, such a method, while more general, would necessarily be approximate, would introduce uncertainty concerning the identity of locations and would generate more paths that are never taken (increasing the number of false negatives in the invariant verification).

A more limited, but simple and precise solution is to *unfold* the symbolic location terms that occur within $f(t_1, \ldots, t_n)$ if their range is a finite set $\{x_1, \ldots, x_n\}$, i.e. to replace any $\langle \text{initial } (g, \overline{y}) \rangle$ occurring within $f(t_1, \ldots, t_n)$ by

$$\texttt{if } \langle \text{initial } (g, \overline{y}) \rangle = x_1 \texttt{ then } x_1 \texttt{ else}$$
$$\ldots$$
$$\texttt{if } \langle \text{initial } (g, \overline{y}) \rangle = x_{n-1} \texttt{ then } x_{n-1} \texttt{ else } x_n.$$

Such case distinctions can then be moved up by the conditional-lifting rules and their branches subsequently simplified by symbolic execution. Modifications of the evaluation rules are needed to avoid unnecessary unfolding and to prevent non-termination due to repeated unfolding of the symbolic location terms in the guards. Another (perhaps more efficient) approach to location unfolding was presented in [11]. Determining the best way to add unfolding to the symbolic execution framework requires further investigation.

Derived and Monitored Functions The symbolic execution rules in Sect. 3 only cover static and controlled functions. *Derived functions* can be seen as macros and expanded, similar to call rules (see Table 5).[24] An approach to deal with *monitored functions* by representing them as uninterpreted functions with an additional *stage* parameter (corresponding to the index i of the state S_i of

[23] The n parameter can also be omitted: then, the number of steps is not limited and the execution (and search for invariant violations) continues until the user stops it.

[24] They are, in fact, implemented in the tool, despite not being described in this paper.

the run) is described in [9]. It is not yet clear whether this approach can serve as a basis for a more systematic treatment of monitored functions.

Simplification of the Symbolic Execution Scheme As mentioned in footnote 18, the symbolic execution scheme may appear cumbersome due to the proliferation of conditional-lifting rules. This could be avoided by changing the representation of the symbolic execution result from a decision tree of nested conditional rules to a flat set of guarded symbolic update sets (G, U). In this way, the need for a case distinction between conditional rules and "symbolic update set rules" $\langle \text{upd } \mathsf{U} \rangle$ would disappear: $\langle \text{upd } \mathsf{U} \rangle$ could simply be represented as $(\langle \text{val } \mathbf{true} \rangle, \mathsf{U})$. Using this representation, the symbolic execution for `par` (as an example) could be defined by a single rule:

$$ {}^{\Pi}[\![R_1 \texttt{ par } R_2]\!]^{\mathsf{S}}_{\eta} = \left\{ ({}^{\Pi}[\![G_1 \wedge G_2]\!]^{\mathsf{S}}_{\eta},\ \mathsf{U}_1 \cup \mathsf{U}_2) \;\middle|\; \begin{array}{l} (G_1, \mathsf{U}_1) \in {}^{\Pi}[\![R_1]\!]^{\mathsf{S}}_{\eta}, \\ (G_2, \mathsf{U}_2) \in {}^{\Pi}[\![R_2]\!]^{\mathsf{S}}_{\eta}, \\ {}^{\Pi}[\![G_1 \wedge G_2]\!]^{\mathsf{S}}_{\eta} \neq \langle \text{val } \mathbf{false} \rangle \end{array} \right\} $$

Note that such a change is not only a matter of notation, but leads to a different symbolic execution algorithm. A drawback of this alternative is that the resulting "flat set of paths" would depart from the structure of the original ASM program and be less human-readable than the "decision tree" rule. On the other hand, the symbolic execution scheme could be defined using fewer rules. It may be worth investigating whether one of the two approaches is more runtime-efficient.

Directions for Future Work The feasibility of the approach to bounded verification presented here has been validated with various experiments, most importantly those published in [9]. Future work should focus on improving its efficiency by applying and adapting state-of-the-art techniques from the fields of symbolic execution, model checking and automated verification.

Furthermore, it could be applied to extend the SMV-based model checking support for ASM [2, 11] to support the `seq` construct by symbolically executing a single step to eliminate `seq` before applying the ASM to SMV mapping.

Finally, an important direction for future research is to extend the scope of the method presented here from mere counterexample-finding to actual proof by combining it with inductive reasoning techniques.

5.3 Conclusions

In this paper, we have presented a symbolic execution framework for sequential ASMs that enables automatic bounded verification backed by SMT solvers. By transforming rules into a simplified decision-tree form, our approach provides both a formal basis for invariant checking and human-readable feedback for model analysis. Future work should focus on covering a broader subset of ASM, improving runtime performance and supporting exhaustive verification in addition to counterexample-finding.

Acknowledgments. Thanks to the anonymous reviewer for the interesting remarks, including the suggestion concerning the simplification of the symbolic execution rules.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Arcaini, P., Bonfanti, S., Dausend, M., Gargantini, A., Mashkoor, A., Raschke, A., Riccobene, E., Scandurra, P., Stegmaier, M.: Unified syntax for abstract state machines. In: Butler, M., Schewe, K.D., Mashkoor, A., Biro, M. (eds.) Abstract State Machines, Alloy, B, TLA, VDM, and Z. pp. 231–236. Springer International Publishing, Cham (2016). https://doi.org/10.1007/978-3-319-33600-8_14
2. Arcaini, P., Gargantini, A., Riccobene, E.: AsmetaSMV: A way to link high-level ASM models to low-level NuSMV specifications. In: Frappier, M., Glässer, U., Khurshid, S., Laleau, R., Reeves, S. (eds.) Abstract State Machines, Alloy, B and Z. pp. 61–74. Springer Berlin Heidelberg, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-11811-1_6
3. Arcaini, P., Gargantini, A., Riccobene, E.: Using SMT for dealing with nondeterminism in ASM-based runtime verification. Electronic Communications of the EASST **70** (Nov 2014). https://doi.org/10.14279/tuj.eceasst.70.970, https://eceasst.org/index.php/eceasst/article/view/2168
4. Arcaini, P., Gargantini, A., Riccobene, E.: SMT for state-based formal methods: the ASM case study. In: Shankar, N., Dutertre, B. (eds.) Automated Formal Methods. Kalpa Publications in Computing, vol. 5, pp. 1–18. EasyChair (2018). https://doi.org/10.29007/djdz, /publications/paper/b6HD
5. Baldoni, R., Coppa, E., Cono D'Elia, D., Demetrescu, C., Finocchi, I.: A survey of symbolic execution techniques. ACM Comput. Surv. **51**(3) (may 2018). https://doi.org/10.1145/3182657, https://doi.org/10.1145/3182657
6. Bombarda, A., Bonfanti, S., Gargantini, A., Riccobene, E., Scandurra, P.: ASMETA tool set for rigorous system design. In: Platzer, A., Rozier, K.Y., Pradella, M., Rossi, M. (eds.) Formal Methods. pp. 492–517. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-71177-0_28
7. Börger, E., Stärk, R.: Abstract State Machines. Springer, Berlin, Heidelberg (2003). https://doi.org/10.1007/978-3-642-18216-7
8. Börger, E.: The Abstract State Machines method for modular design and analysis of programming languages. Journal of Logic and Computation **27**(2), 417–439 (2017). https://doi.org/10.1093/logcom/exu077
9. Braghin, C., Del Castillo, G., Riccobene, E., Valentini, S.: Using symbolic model execution to detect vulnerabilities of smart contracts. In: Leuschel, M., Ishikawa, F. (eds.) 11th Int. Conf. on Rigorous State-Based Methods, ABZ 2025. Lecture Notes in Computer Science, vol. 15728, pp. 31–51. Springer (2025). https://doi.org/10.1007/978-3-031-94533-5_3
10. Del Castillo, G.: Using Symbolic Execution to Transform Turbo Abstract State Machines into Basic Abstract State Machines. In: 10th Int. Conf. on Rigorous State-Based Methods, ABZ 2024. Lecture Notes in Computer Science, vol. 14759, pp. 215–222. Springer (2024). https://doi.org/10.1007/978-3-031-63790-2_15

11. Del Castillo, G., Winter, K.: Model checking support for the ASM high-level language. In: Graf, S., Schwartzbach, M. (eds.) Tools and Algorithms for Construction and Analysis of Systems (TACAS 2000). pp. 331–346. Springer, Berlin, Heidelberg (2000). https://doi.org/10.1007/3-540-46419-0_23
12. Dorfmeister, D., Ferrarotti, F., Fischer, B., Haslinger, E., Ramler, R., Zimmermann, M.: An approach for safe and secure software protection supported by symbolic execution. In: Kotsis, G., Tjoa, A.M., Khalil, I., Moser, B., Mashkoor, A., Sametinger, J., Khan, M. (eds.) Database and Expert Systems Applications - DEXA 2023 Workshops. pp. 67–78. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-39689-2_7
13. Gargantini, A., Riccobene, E., Scandurra, P.: A metamodel-based language and a simulation engine for abstract state machines. JUCS - Journal of Universal Computer Science **14**(12), 1949–1983 (2008). https://doi.org/10.3217/jucs-014-12-1949, https://doi.org/10.3217/jucs-014-12-1949
14. Glesner, S., Goos, G., Zimmermann, W.: Verifix: Konstruktion und Architektur verifizierender Übersetzer (Verifix: Construction and architecture of verifying compilers). it - Information Technology **46**(5), 265–276 (2004). https://doi.org/10.1524/itit.46.5.265.44799
15. King, J.C.: Symbolic execution and program testing. Commun. ACM **19**(7), 385–394 (jul 1976). https://doi.org/10.1145/360248.360252
16. Lezuo, R.: Scalable Translation Validation. Ph.D. thesis, Vienna University of Technology (2014), https://www.complang.tuwien.ac.at/tbfg
17. Ma, G.Z.S.: Model checking support for CoreASM: model checking distributed abstract state machines using Spin. Master's thesis, Simon Fraser University (2007), https://summit.sfu.ca/item/8056
18. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008). pp. 337–340. Springer Berlin Heidelberg, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24
19. Paun, V.A., Monsuez, B., Baufreton, P.: Integration of symbolic execution into a formal abstract state machines based language. IFAC-PapersOnLine **50**(1), 11251–11256 (2017). https://doi.org/10.1016/j.ifacol.2017.08.1610, 20th IFAC World Congress
20. Schellhorn, G., Ahrendt, W.: The WAM case study: verifying compiler correctness for Prolog with KIV. In: Bibel, W., Schmitt, P.H. (eds.) Automated deduction - a basis for applications: volume 3 - applications. Kluwer Academic Publishers (1998)
21. Schellhorn, G., Ernst, G., Pfähler, J., Bodenmüller, S., Reif, W.: Symbolic execution for a clash-free subset of ASMs. Science of Computer Programming **158**, 21–40 (2018). https://doi.org/10.1016/j.scico.2017.08.014
22. Topor, R.W.: Interactive program verification using virtual programs. Ph.D. thesis, University of Edinburgh (1975), https://era.ed.ac.uk/handle/1842/6610
23. Zimmermann, W., Weißbach, M.: A framework for modeling the semantics of synchronous and asynchronous procedures with abstract state machines. Logic, Computation and Rigorous Methods: Essays Dedicated to Egon Börger on the Occasion of His 75th Birthday pp. 326–352 (2021)